

LAB ASSIGNMENT-5

Name- Rajat Rao

Roll no-2401201067

Course-BCA(AI&DS)

Input-

```
1  import java.io.*;
2  import java.util.*;
3
4  abstract class Person {
5      protected String name;
6      protected String email;
7      public Person(String name, String email) {
8          this.name = name;
9          this.email = email;
10         void Person.displayInfo()
11     }
12     public abstract void displayInfo();
13 }
14
15 class Student extends Person {
16     private int rollNo;
17     private String course;
18     private double marks;
19     private String grade;
20
21     public Student(int rollNo, String name, String email, String course, double marks) {
22         super(name, email);
23         this.rollNo = rollNo;
24         this.course = course;
25         this.marks = marks;
26         calculateGrade();
27     }
28
29     public int getRollNo() { return rollNo; }
```

```

29     public String getName() { return name; }
30     public double getMarks() { return marks; }
31
32     public void setCourse(String course) { this.course = course; }
33     public void setMarks(double marks) { this.marks = marks; calculateGrade(); }
34
35     public void calculateGrade() {
36         if (marks >= 90) grade = "A";
37         else if (marks >= 75) grade = "B";
38         else if (marks >= 60) grade = "C";
39         else grade = "D";
40     }
41
42     public void displayInfo() {
43         System.out.println("Roll No: " + rollNo);
44         System.out.println("Name: " + name);
45         System.out.println("Email: " + email);
46         System.out.println("Course: " + course);
47         System.out.println("Marks: " + marks);
48         System.out.println("Grade: " + grade);
49     }
50
51     public String toFileString() {
52         return rollNo + "," + name + "," + email + "," + course + "," + marks;
53     }
54 }
55

```

```

56 interface RecordActions {
57     void addStudent(Student s) throws Exception;
58     void deleteStudent(String name) throws Exception;
59     void searchStudent(String name) throws Exception;
60     void updateStudent(int rollNo, double marks) throws Exception;
61     void viewAllStudents();
62 }
63
64 class StudentNotFoundException extends Exception {
65     public StudentNotFoundException(String msg) { super(msg); }
66 }
67
68 class InvalidInputException extends Exception {
69     public InvalidInputException(String msg) { super(msg); }
70 }
71
72 class Loader implements Runnable {
73     public void run() {
74         try {
75             for (int i = 0; i < 3; i++) {
76                 System.out.print(s: "Loading.");
77                 Thread.sleep(millis: 500);
78             }
79             System.out.println();
80         } catch (InterruptedException e) {
81             Thread.currentThread().interrupt();
82         }
83     }
84 }

```

```

84     }
85
86     class StudentManager implements RecordActions {
87         private List<Student> students = new ArrayList<>();
88         private Map<Integer, Student> studentMap = new HashMap<>();
89         private final String fileName = "students.txt";
90
91         public StudentManager() {
92             loadFromFile();
93         }
94
95         public void addStudent(Student s) throws Exception {
96             if (studentMap.containsKey(s.getRollNo())) throw new InvalidInputException(msg: "Duplicate Roll Number");
97             students.add(s);
98             studentMap.put(s.getRollNo(), s);
99         }
100
101         public void deleteStudent(String name) throws Exception {
102             Iterator<Student> it = students.iterator();
103             boolean found = false;
104             while (it.hasNext()) {
105                 Student s = it.next();
106                 if (s.getName().equalsIgnoreCase(name)) {
107                     it.remove();
108                     studentMap.remove(s.getRollNo());
109                     found = true;
110                     break;

```

```

111             }
112         }
113         if (!found) throw new StudentNotFoundException(msg: "Student not found");
114     }
115
116     public void searchStudent(String name) throws Exception {
117         for (Student s : students) {
118             if (s.getName().equalsIgnoreCase(name)) {
119                 s.displayInfo();
120                 return;
121             }
122         }
123         throw new StudentNotFoundException(msg: "Student not found");
124     }
125
126     public void updateStudent(int rollNo, double marks) throws Exception {
127         Student s = studentMap.get(rollNo);
128         if (s == null) throw new StudentNotFoundException(msg: "Student not found");
129         s.setMarks(marks);
130     }
131
132     public void viewAllStudents() {
133         Iterator<Student> it = students.iterator();
134         while (it.hasNext()) it.next().displayInfo();
135     }
136

```

```

137     public void sortByMarks() {
138         students.sort(Comparator.comparingDouble(Student::getMarks).reversed());
139     }
140
141     public void saveToFile() {
142         try (BufferedWriter bw = new BufferedWriter(new FileWriter(fileName))) {
143             for (Student s : students) {
144                 bw.write(s.toFileString());
145                 bw.newLine();
146             }
147         } catch (IOException e) {
148             System.out.println(x: "Error saving file");
149         }
150     }
151
152     private void loadFromFile() {
153         File file = new File(fileName);
154         if (!file.exists()) return;
155         try (BufferedReader br = new BufferedReader(new FileReader(file))) {
156             String line;
157             while ((line = br.readLine()) != null) {
158                 String[] p = line.split(regex: ",");
159                 Student s = new Student(Integer.parseInt(p[0]), p[1], p[2], p[3], Double.parseDouble(p[4]));
160                 students.add(s);
161                 studentMap.put(s.getRollNo(), s);
162             }
163         } catch (Exception e) {

```

```

164             System.out.println(x: "Error loading file");
165         }
166     }
167 }
168
169 public class StudentRecordSystem {
170     Run | Debug
171     public static void main(String[] args) {
172         Scanner sc = new Scanner(System.in);
173         StudentManager manager = new StudentManager();
174         int choice;
175
176         do {
177             System.out.println(x: "===== Capstone Student Menu =====");
178             System.out.println(x: "1. Add Student");
179             System.out.println(x: "2. View All Students");
180             System.out.println(x: "3. Search by Name");
181             System.out.println(x: "4. Delete by Name");
182             System.out.println(x: "5. Sort by Marks");
183             System.out.println(x: "6. Save and Exit");
184             System.out.print(s: "Enter choice: ");
185             choice = sc.nextInt();
186
187             try {
188                 Thread t = new Thread(new Loader());
189                 t.start();

```

```

189         t.join();
190
191         switch (choice) {
192             case 1:
193                 System.out.print(s: "Enter Roll No: ");
194                 int r = sc.nextInt();
195                 sc.nextLine();
196                 System.out.print(s: "Enter Name: ");
197                 String n = sc.nextLine();
198                 System.out.print(s: "Enter Email: ");
199                 String e = sc.nextLine();
200                 System.out.print(s: "Enter Course: ");
201                 String c = sc.nextLine();
202                 System.out.print(s: "Enter Marks: ");
203                 double m = sc.nextDouble();
204                 manager.addStudent(new Student(r, n, e, c, m));
205                 break;
206             case 2:
207                 manager.viewAllStudents();
208                 break;
209             case 3:
210                 sc.nextLine();
211                 System.out.print(s: "Enter name to search: ");
212                 manager.searchStudent(sc.nextLine());
213                 break;
214             case 4:

```

```

215                 sc.nextLine();
216                 System.out.print(s: "Enter name to delete: ");
217                 manager.deleteStudent(sc.nextLine());
218                 System.out.println(x: "Student record deleted.");
219                 break;
220             case 5:
221                 manager.sortByMarks();
222                 manager.viewAllStudents();
223                 break;
224             case 6:
225                 manager.saveToFile();
226                 System.out.println(x: "Saved and exiting");
227                 break;
228             default:
229                 System.out.println(x: "Invalid choice");
230         }
231     } catch (Exception ex) {
232         System.out.println(ex.getMessage());
233     }
234 } while (choice != 6);
235 }
236 }

```

```
===== Capstone Student Menu =====
```

1. Add Student
2. View All Students
3. Search by Name
4. Delete by Name
5. Sort by Marks
6. Save and Exit

```
Enter choice: 1
```

```
Loading.Loading.Loading.
```

```
Enter Roll No: 67
```

Output- Enter Name: Rajat Rao

```
Enter Name: Rajat Rao
```

```
Enter Email: rajatrao404@gmail.com
```

```
Enter Course: BCA(AI&DS)
```

```
Enter Marks: 85
```

```
===== Capstone Student Menu =====
```